Choosing an Optimisation Method for Severity-Ladder Design

Review copy. Highlighted boxes below flag specific points for review (**REVIEW**) and open decisions for future work (**DECISION**). Section 9 at the end collects items that do not attach to one specific location. Remove every `\reviewnote`/`\decisionnote` call (and their definitions, near the top of the source) before final submission.

1 Introduction

Phase 3 of this work asks: for k degradation levels, what severities $\chi_2, \ldots, \chi_{k-1}$ should trigger levels $L_2, \ldots, L_{k-1}$ (severity-indexed response time and admissibility, task_model.tex), and what optimisation method should be used to find them? The method cannot be chosen from habit or convenience – the wrong choice either wastes evaluation budget the problem does not need spent, or under-samples a domain that needs full coverage. This document characterises the search problem first (Section 2), then uses that characterisation to assess the candidate optimisation techniques (Section 3), states the resulting hypothesis (Section 4), and proposes the staged empirical protocol that tests it (Section 5) – itself already partly executed, not merely proposed, as the pilots in Section 5 show.

2 Problem Characterisation: The Optimisation Landscape

2.1 Decision Variables and Domain

A k-level severity ladder has $k-1$ trigger points, but the deepest, χ_{k-1}, is not free: the safety argument (admissible drop sets, task_model.tex) requires $\chi_1 = 0$, and pinning χ_{k-1} recovers the classic two-level guarantee exactly at the deepest level. The optimisation is therefore over $\mathbf{k-2}$ free variables $\chi_2 \le \chi_3 \le \cdots \le \chi_{k-2}$, each bounded in $[0, 1]$ and subject to a total order. This is small in every case this work considers:

Table 1: Free dimensions and feasible-set size by level count

k	free dimensions	unordered hypercube ($g = 20$)	ordered feasible set
3	1	400	**20**
4	2	8,000	**210**
5	3	160,000	**1,540**

The right-hand column is not the hypercube g^{k-2} but the number of non-decreasing $(k-2)$-tuples drawn from a grid of g values, $\binom{g+k-3}{k-2}$ (a standard multiset/"stars-and-bars" count): the ordering constraint is not an incidental detail but the dominant factor in how large the space actually is. A search method that samples the unconstrained box and repairs infeasible points by sorting – the natural approach for a population-based or surrogate-based method operating on a free vector – wastes evaluations on points that collapse onto the same feasible tuple, and in the worst case biases the sampling measure over the feasible region without the practitioner intending it to.

2.2 The Objective is a Noisy, Expensive-per-Sample Simulator Output

Φ (metrics_objective.md) is not observed directly; it is estimated from a discrete-event simulation whose execution times and HI-criticality occurrences are drawn at random. Per-seed coefficients of variation, measured on this simulator: 0.157 (NiD), 0.314 (JNE), 0.199 (degraded ticks). Neither the objective nor its

gradient is available in closed form, so the landscape is a black box in the sense every method below has to contend with, not merely difficult to solve analytically.

A single simulation is cheap (≈ 43 ms for 10^6 ticks) and embarrassingly parallel across seeds and task sets, but the noise above means one evaluation at one lattice point is not one sample – it is a paired mean over a seed block, and the block has to be large enough to resolve the effect size the study cares about (§3 returns to what this implies for methods that spend their budget on *many* points rather than *repeated* samples at few).

2.3 What Is Not Yet Known

Whether Φ is unimodal, smooth, or has a plateau over the severity domain is an open empirical question, not an assumption available to lean on. A method whose efficiency depends on such structure – surrogate modelling, gradient estimation, population convergence – is therefore choosing to bet on a property this document cannot yet certify. A method that does not depend on it is not making that bet, at the cost of not exploiting the structure if it does turn out to hold. This tension is the crux of Section 3, and Section 5 answers it empirically rather than leaving it open.

3 Related Work: Optimisation Techniques for Expensive Stochastic Objectives

Optimising the parameters of a stochastic simulation, rather than a closed-form function, is the subject of *simulation optimisation*, whose central tension Fu [1] frames precisely: theoretical convergence results are proven for algorithms that are rarely what practitioners actually run, while the general-purpose metaheuristics that are widely used borrow convergence properties from deterministic optimisation that do not transfer cleanly to a noisy objective. Four families of technique are relevant to the problem in Section 2.

3.1 Exhaustive Search and Design of Experiments

Design of Experiments (DOE) fits a statistical model – typically linear or low-order polynomial – to a systematically chosen sample of the parameter space, trading exhaustive coverage for a smaller, structured design (e.g. a factorial or Latin-hypercube design) when the space is too large to enumerate. Tate et al. [6] apply exactly this trade-off to a directly comparable problem: tuning multiple sensornet protocol parameters against several competing performance metrics, a genuinely high-dimensional, combinatorially explosive space where full enumeration is not an option. Their DOE fit yielded a more generally applicable model of the parameter space; a competing evolutionary search (Section 3.2) found a broader range of viable solutions at lower cost for the specific instance evaluated. That finding does not transfer here, and the reason it does not is itself informative: their conclusion favours evolutionary search precisely where full enumeration is infeasible, and Section 2 establishes that the severity ladder is not in that regime – 1,540 points at $k = 5$ is exhaustively enumerable, so DOE's usual purpose, choosing which subset of a space to sample because the whole space cannot be, does not arise. What remains applicable is the weaker, unconditional claim: over a space small enough to enumerate, exhaustive search dominates any sampling design of it, since a sample is only ever a strict subset of what enumeration already covers, at correspondingly less information per unit of the region actually examined.

3.2 Evolutionary and Population-Based Metaheuristics

Genetic algorithms [2, 7] and related population-based metaheuristics such as simulated annealing [4] search by iteratively perturbing and selecting among a working set of candidate solutions, and are the default choice when a space is too large or too irregular to enumerate and no exploitable structure is known in advance – exactly the case in Tate et al.'s protocol-tuning problem above. Applied here, the same population-based iteration that gives these methods their reach becomes a liability: each generation adds evaluations, and Section 2's per-seed noise means every added evaluation is a further noisy draw. Selecting the best of M such draws is optimistically biased by $\sigma\sqrt{2\ln M}$ (a standard result for the expected maximum of M noisy samples).

A population of 50 run for 100 generations – an unremarkable configuration – makes 5,000 evaluations; at this simulator's measured noise, that inflates the reported optimum's bias to roughly 12%, more than double the 5% practical-significance threshold this work uses to call a result meaningful (metrics_objective.md). *A genetic algorithm run against a perfectly flat objective would return a publishable-looking optimum.* This is a general property of any method whose evaluation count scales with iterations rather than with the size of the space being covered, and it argues against every method in this family for a domain this small, independent of how well-suited any of them are in a domain too large to enumerate.

3.3 Bayesian Optimisation and Surrogate Models

Bayesian optimisation [3, 5] fits a probabilistic surrogate – typically a Gaussian process – to the evaluations seen so far, and chooses the next point by an acquisition function that balances exploiting the surrogate's optimum against exploring where it is most uncertain. It is the standard choice when each evaluation is expensive enough that minimising the *number* of evaluations dominates every other concern, and its sample efficiency is real precisely under that condition. It is not the condition here: Section 2 costs a single simulation at tens of milliseconds and the whole lattice at a few CPU-hours, so there is no evaluation budget for a surrogate to conserve. Two further mismatches follow from Section 2 specifically. First, the deliverable is a response surface, not an optimum – Bayesian optimisation is built to concentrate samples near a believed optimum, which is the opposite of what characterising a surface requires, and recovering the surface afterwards needs a further, separately budgeted sampling pass. Second, the surrogate assumes a smoothness the objective has not yet been shown to have (Section 2); a Gaussian-process surrogate additionally needs an explicit noise term fitted correctly to the measured heteroscedastic variance above, which is one more unverified assumption riding on an already-unverified one.

3.4 Gradient-Based and Derivative-Free Local Search

Classical gradient-based optimisation is inapplicable outright: Φ is the output of a discrete-event simulation with no closed form and no analytic derivative, and finite-difference gradient estimates would themselves be noisy at the measured coefficients of variation, compounding rather than resolving the problem Section 3.2 raises. Derivative-free local search (pattern search, Nelder–Mead) removes the need for a derivative but keeps the core liability: it is iterative and local, so it inherits both the winner's curse of repeated noisy evaluation and a dependence on the smoothness this document has not established.

4 Working Hypothesis, and Why It Is Not Yet a Conclusion

Exhaustive enumeration of the ordered severity lattice (Section 2), evaluated under common random numbers. Every technique surveyed in Section 3 earns its place by trading coverage for sample efficiency in a large or unstructured space; the severity ladder's space is neither. DOE's usual justification – covering a space too large to enumerate – does not apply once the ordering constraint is respected directly rather than repaired after the fact. Evolutionary and Bayesian methods both spend evaluations to save evaluations, at a specific, quantified cost (winner's-curse bias, or an unrepresentative sample of the surface) that this domain does not need paid. Enumeration additionally reports the whole surface natively, has one hyperparameter – grid resolution – against a genetic algorithm's at least eight or a Gaussian process's comparable number, and needs no additional dependency.

That argument is structural: it follows from counting the feasible set and from the measured cost of one evaluation, and it does not depend on knowing Φ's shape. It is not, on its own, an empirical result. It has not been checked against a real comparison between methods, and the pilot in Section 5 shows at least one place a plausible intuition about this problem turned out to be wrong when actually measured (Section 7.9) – which is reason enough not to extend the same trust to an argument that has not been tested at all. Section 5 is the protocol that tests it, structured so that if the hypothesis is wrong, that becomes visible before it is load-bearing for the rest of this work, rather than after.

Whichever method the protocol below settles on, it must respect the noise floor Section 2 establishes: configurations are compared as paired samples under common random numbers, never as independent means, since an unpaired comparison at this simulator's measured noise needs roughly two orders of magnitude more

task sets to resolve the same effect (metrics_objective.md, evaluation_methodology.tex §3). This is the one requirement every method in Section 3 shares, and the one a choice of *method* cannot substitute for.

Retrospective note. The hypothesis is confirmed, but not by the comparison this section anticipated running. Section 7.3 shows exhaustive search's own upper bound on what any method could win over the default configuration is 0.79% – so the reason enumeration was the right choice is not that it beat a genetic algorithm in a head-to-head test, but that the prize on offer was never large enough for the choice of method to matter. That is a stronger vindication of the structural argument above than a head-to-head win would have been: it is not that enumeration happened to find a better point, it is that no method could have found a materially better one.

5 A Staged Empirical Protocol

Choosing a method well needs three things a one-shot argument cannot supply: a measured landscape to reason from instead of an assumed one, a real comparison between candidate methods rather than a critique of each in isolation, and a evaluation budget sized to the noise actually observed rather than picked by convention. The protocol below produces each in turn, each phase's output gating the next. Two of the three phases already have a worked pilot with real data, run on the infrastructure that exists today (Section 2's severity-indexed response time and the two-level engine); the third needs the k-level engine that does not yet exist and is specified precisely so it is ready once that engine does.

6 Revised Protocol

The protocol in Sections 7.7 to 7.9 was designed before the k-level engine existed, and assumed that trigger spacing was the quantity worth optimising. With the engine built and verified, a structural pilot contradicts that assumption outright, and the protocol has to be rebuilt around what the pilot found rather than around what was expected.

6.1 What the structural pilot found

All figures below are paired (identical task sets and seed blocks), aggregated one value per task set, and reported against this study's declared 5% practical-significance threshold. $n = 12$, $U = 0.7$, $CF = 2$, $f_p = 0.2$, 200,000 ticks.

Table 2: Effect of each design knob on service ratio

Knob	Effect	Resolvable at	Verdict
Scheme vs. two-level AMC	+5.5 to +11.9%	—	**above threshold**
Progressive vs. shed-early	+0.24 to +0.62%	0.25–0.79%	1 of 4 resolved
Shedding order	+0.96 to +1.54%	1.2–1.4%	1 of 2 resolved
Greedy vs. exhaustive optimum	+0.29 to +0.83%	0.3–1.0%	not resolved
Severity spacing (shed-early)	**exactly** 0	—	structurally vacuous
$k \in \{2, 3, 4, 5\}$ (shed-early)	**exactly** 0	—	structurally vacuous
x_{LO}	≈ 0, slightly negative	—	settled negative

Two entries are exact zeros rather than small numbers, and the reason is structural rather than statistical. `drop_set_shed_early` requires one drop set feasible at *every* operating severity, and the deepest is pinned to 1; by monotonicity (M1) feasibility there implies it everywhere below, so the intermediate severities never bind and the drop set is identical for every spacing – confirmed on 16/16 task sets. Since operating budgets are an analysis quantity and not a run-time throttle (see the task model's "Execution Budget Enforcement"), the drop set is the *only* thing that distinguishes one level from another. Identical drop sets therefore make the levels observationally identical, and k has no effect: service ratio agrees to six decimal places across $k \in \{2, 3, 4, 5\}$.

Consequence. Under shed-early the k-level ladder collapses to two levels, and the entire gain over two-level AMC comes from *shedding a subset rather than everything* – not from grading. Grading only becomes observable under a *progressive* ladder, where $S_1 \subsetneq S_{k-1}$, and there it is real: a progressive ladder at severities $(0, 0.25)$ sheds 14.6% at the shallowest rung under the conservative point and 0.0% under the termination point, against shed-early's 69.4% and 57.6%. That is the original promise of graded degradation – mild overruns needing no shedding at all – and it is the one configuration in which the multi-level structure earns its keep.

The cost is feasibility, and it is the headline risk. A progressive ladder does not always exist: 16/24 task sets at severities $(0, 0.25)$, and only 6–7/24 at $(0, 0.5)$, because a task first shed at a deep rung has an infinite shed instant for 33% of HI-criticality tasks at $\chi = 1$. Shed-early exists always. Applicability, not optimality, is therefore the quantity the study should be measuring.

6.2 Why the original Phase B is retired

Phase B asked which optimisation method finds the best trigger spacing under a matched budget. Trigger spacing has *no effect at all* under the policy the carry-in analysis endorses, so the question is vacuous there; and where it is not vacuous, every knob in Table 2 moves the objective by under 1.6%, against a declared threshold of 5%. The whole answer space of the method comparison lies an order of magnitude below what this study calls a meaningful difference. Running a genetic algorithm against enumeration to compete for tenths of a percent would produce a number, not a finding.

The exhaustive optimum is itself only 0.29–0.83% better than a greedy shedding order, and that gap is not resolvable at the pilot's scale. This retires the metaheuristic question *with evidence* rather than by assertion, which is what Phase B existed to do – it simply answers it in the negative and far more cheaply than the original design would have.

6.3 The replacement: four stages

Stage 0 – structural pruning. Before measuring anything, establish which knobs can have an effect. Costs minutes, and it is what revealed the two exact zeros above. Must be re-run whenever the execution model changes, because both zeros are consequences of the model rather than of the task sets.

Stage 1 – feasibility characterisation. For each configuration (operating point × policy × severity vector), what fraction of task sets admit a ladder at all? This is a design-time property needing no simulation, so it is cheap, and it is the primary applicability result: a configuration that wins by 6% on a quarter of task sets is a different proposition from one that wins by 0.6% on all of them.

Stage 2 – regime map. The main experiment, and the only one whose effects exceed threshold. Service ratio against two-level AMC across $(U, D_i/T_i, CF, f_p)$, for both operating points, reporting where the scheme wins, by how much, and *where it loses* – at $U = 0.85$ with tight deadlines it already loses by 0.25 percentage points, and that boundary is a result, not an embarrassment.

Stage 3 – configuration selection under a bounded-gain budget. Since every knob is sub-threshold, the question is not "find the optimum" but "is the simplest configuration good enough, and by how much could anything beat it?" Deliverable: an upper bound on what any search could win, obtained by exhaustive enumeration on small task sets. If that bound is below threshold – as the pilot suggests – the simplest configuration is selected and the matter is closed.

Stage 4 – budget calibration, per operating point. The two points do not behave alike and must be sized separately: the shedding-order effect is resolved under the conservative point (+1.54% against 1.42% resolvable) and not under the termination point (+0.96% against 1.17%). Sizing one study from the other's pilot would under-power it.

7 Stage Results

Every number below is produced by `amc_tasksim.experiments.multilevel_protocol`, tested in `tests/experiments/test_` and reproducible from the module's public functions – this section is a report of what running them found, not a hand-computed table.

7.1 Stage 1: Feasibility

Forty task sets per cell ($n = 10$, $CP = 0.5$, $CF = 2$), swept over utilisation, deadline tightness, operating point and severity gap. No simulation is required – feasibility is decided by response-time analysis alone.

Table 3: Feasibility at the most permissive progressive spacing, $(0, 0.25)$. $P(\cdot)$ is the fraction of task sets admitting the ladder; shed% is the mean percentage of LO-criticality tasks shed at the shallowest rung, over task sets where the ladder exists.

U	Regime	Point	n	P(shed-early)	P(progressive)	shed%(progressive)
0.6	implicit	A	40	1.00	0.90	10.6%
0.6	implicit	B	40	1.00	0.90	0.0%
0.6	tight	A	40	1.00	0.72	43.4%
0.6	tight	B	40	1.00	0.68	0.0%
0.7	implicit	A	40	1.00	0.70	17.9%
0.7	implicit	B	40	1.00	0.65	0.0%
0.7	tight	A	40	1.00	0.65	46.2%
0.7	tight	B	40	1.00	0.60	0.0%
0.8	implicit	A	40	1.00	0.60	27.5%
0.8	implicit	B	40	1.00	0.57	0.0%
0.8	tight	A	40	1.00	0.57	64.3%
0.8	tight	B	40	1.00	0.50	0.0%
0.9	implicit	A	40	1.00	0.70	57.1%
0.9	implicit	B	40	1.00	0.65	0.0%
0.9	tight	A	12	1.00	0.67	77.5%
0.9	tight	B	12	1.00	0.58	0.0%

Three findings, in order of how much they change the earlier picture.

Shed-early is feasible everywhere, always ($P = 1.00$). Expected: shedding everything at R_i^{lo} is AMC-rtb's own equation, so shed-early is certifiable exactly when the task set is AMC-rtb-schedulable, which is the population's qualification criterion.

Progressive feasibility falls as the regime hardens. From 90% ($U = 0.6$, implicit) to as low as 50% ($U = 0.8$, tight, point B) and the population itself grows scarce at $U = 0.9$ tight ($40 \to 12$ candidates found in the same search budget). This is the applicability cost Section 7.7's successor was designed to surface: a progressive ladder is not a free upgrade over shed-early, it is a bet that may not be available when the regime is hardest – precisely where the gain would matter most.

Under Point B, the shallowest rung sheds nothing wherever it is feasible. shed%(progressive) is exactly 0.0% in every Point-B row at this spacing. This is not an averaging artefact: every task set that admits a progressive ladder at $(0, 0.25)$ under the termination point does so while shedding zero LO-criticality tasks at L_1 – full retention, right up until the deeper rung. Under Point A the same column ranges from 10.6% to 77.5%, because clause 2's per-rung certification is the binding constraint and it tightens as U and deadline pressure rise.

Severity spacing is not vacuous for progressive ladders – but it is close to vacuous for *service*. Section 6.1 showed spacing has no effect at all under shed-early. Under a progressive ladder it is not vacuous in this narrower sense: L_1's *operating* severity is `severities`[1], so widening the first gap directly raises what L_1 is charged and, correspondingly, how much it must shed to stay certified. At $U = 0.7$, implicit, point A: shed%(progressive) at L_1 rises from 17.9% at spacing $(0, 0.25)$ to 46.7% at $(0, 0.5)$ to 64.0% at $(0, 0.75)$, while feasibility falls from 70% to 38% to 38%. But a paired check of whether this moves *service ratio* (not just shed-set size), on the 30-task-set pilot restricted to sets feasible under both spacings compared, resolved in only 1 of 6 comparisons tested (point B, $(0, 0.25) \to (0, 0.5)$, +0.58%, $n = 8$) – everywhere else the difference was not distinguishable from zero at that sample size. Spacing controls what is *shed*, robustly; whether that translates into *delivered service* is a smaller and noisier effect, consistent with every other sub-threshold knob in Table 2.

7.2 Stage 2: The Regime Map

Thirty task sets per (U, regime) cell ($n = 10$, $CP = 0.5$, $CF = 2$, $f_p = 0.2$, 200,000 ticks, 8 seeds), paired against two-level AMC-RA computed on the same task sets and seeds. Progressive uses severities $(0, 0.25)$, the most feasible spacing found in Stage 1.

Table 4: Regime map, implicit deadlines ($D_i = T_i$). $n = 10$, $CP = 0.5$, $CF = 2$, $f_p = 0.2$, 200,000 ticks $\times$ 8 seeds. Progressive severities $(0, 0.25)$. 'Point' is the operating point (A conservative, B termination); 'Scheme' is the scheme's service ratio; 'Diff' and 'Rel.' are the paired difference from AMC-RA; 'Resolvable' is the smallest relative effect this sample could distinguish from zero; 'Verdict' follows contract.PairedResult.

U	Point	Policy	n	AMC-RA	Scheme	Diff	Rel.	Resolvable	Verdict
0.60	A	shed early	30	0.9426	0.9729	+0.0303	+3.21%	1.61%	sig.
0.60	A	progressive	28	0.9430	0.9790	+0.0361	+3.83%	1.64%	sig.
0.60	B	shed early	30	0.9426	0.9848	+0.0422	+4.47%	2.06%	sig.
0.60	B	progressive	28	0.9430	0.9890	+0.0461	+4.89%	2.10%	sig.
0.70	A	shed early	30	0.9074	0.9445	+0.0371	+4.09%	2.58%	sig.
0.70	A	progressive	20	0.8909	0.9532	+0.0623	+6.99%	3.60%	**practsig**
0.70	B	shed early	30	0.9074	0.9591	+0.0517	+5.69%	3.23%	**practsig**
0.70	B	progressive	18	0.8938	0.9776	+0.0838	+9.38%	4.82%	**practsig**
0.80	A	shed early	30	0.8678	0.9017	+0.0339	+3.91%	2.89%	sig.
0.80	A	progressive	20	0.8515	0.9246	+0.0731	+8.58%	4.12%	**practsig**
0.80	B	shed early	30	0.8678	0.9283	+0.0605	+6.97%	3.68%	**practsig**
0.80	B	progressive	19	0.8490	0.9520	+0.1030	+12.14%	4.98%	**practsig**
0.90	A	shed early	30	0.7401	0.7878	+0.0477	+6.44%	4.31%	**practsig**
0.90	A	progressive	22	0.7331	0.8211	+0.0881	+12.01%	5.04%	**practsig**
0.90	B	shed early	30	0.7401	0.8441	+0.1040	+14.05%	6.77%	**practsig**
0.90	B	progressive	20	0.7307	0.8936	+0.1629	+22.29%	8.19%	**practsig**

The scheme beats AMC-RA in every one of the 32 cells tested. Every single diff in Tables 4 and 5 is positive, and 22 of 32 are practically significant by this study's own threshold. Service gains widen as the regime hardens: from +2.4% ($U = 0.6$, tight, point A, shed-early) up to +27.4% ($U = 0.9$, tight, point B, progressive). Progressive beats shed-early in every cell where both are feasible, and Point B beats Point A in every cell – both orderings are exactly what task_model.tex's "Mode Transition Protocol" and "Shed-Aware Response Time" sections predict, now confirmed across the grid rather than on isolated pilots.

A methodological correction, reported because it happened. An earlier version of this analysis reported the scheme *losing* to AMC-RA at $U = 0.85$, tight deadlines, by 0.25 percentage points (0.6702 against 0.6727), computed from a 25–30-task-set, 3-seed pilot. That result does not reproduce. Re-measured independently with a fresh 50-task-set population at the same regime, 10 seeds, 300,000 ticks: Point A scores 0.7195 against AMC-RA's 0.7074 (+1.70%, 95% CI [−0.10%, +2.51%], not resolved – the interval cannot rule out a very small loss, but overwhelmingly favours a small gain over a loss); Point B scores 0.7489 (+5.87%, 95% CI [+1.75%, +6.55%], resolved and practically significant). Neither result supports the original claim of an outright loss.

The likely cause is exactly what Section 7.9 (Stage 4's predecessor) exists to prevent: $U = 0.85$ with tight deadlines sits in a scarce region of the population (Section 7.1), and a 3-seed, ∼25-task-set sample is not powered to distinguish a small effect from noise in a scarce, high-variance regime – the sample's own `resolvable` bound was never checked against `EFFECT_FLOOR` before the claim was written down. This is retained here, rather than silently corrected, as the concrete case for running Stage 4 before trusting a small pilot: the failure mode it warns against is not hypothetical, it happened in this project's own paper.

Consequence for task_model.tex's "Two Operating Points" section. The claim that "both points degenerate in the hardest regime tested" and that "neither point should be recommended unconditionally" should be read as superseded by this section. Across the full grid tested here, including the specific

Table 5: Regime map, tight deadlines (D_i at 30% of the way from R_i^{lo} to T_i). Same configuration as Table 4. n shrinks at $U = 0.90$ because qualifying task sets at this combination of high utilisation and tight deadlines are scarce (Section 7.1).

U	Point	Policy	n	AMC-RA	Scheme	Diff	Rel.	Resolvable	Verdict
0.60	A	shed early	30	0.9046	0.9263	+0.0217	+2.40%	1.89%	sig.
0.60	A	progressive	21	0.8886	0.9319	+0.0433	+4.87%	2.54%	sig.
0.60	B	shed early	30	0.9046	0.9364	+0.0318	+3.51%	1.93%	sig.
0.60	B	progressive	19	0.8883	0.9488	+0.0605	+6.81%	2.59%	**practsig**
0.70	A	shed early	30	0.8231	0.8658	+0.0427	+5.19%	2.74%	**practsig**
0.70	A	progressive	21	0.8301	0.8951	+0.0650	+7.83%	3.30%	**practsig**
0.70	B	shed early	30	0.8231	0.8884	+0.0654	+7.94%	3.18%	**practsig**
0.70	B	progressive	19	0.8215	0.9246	+0.1032	+12.56%	3.78%	**practsig**
0.80	A	shed early	30	0.7542	0.7770	+0.0228	+3.03%	2.89%	sig.
0.80	A	progressive	18	0.7453	0.7897	+0.0443	+5.95%	3.35%	**practsig**
0.80	B	shed early	30	0.7542	0.8116	+0.0575	+7.62%	4.66%	**practsig**
0.80	B	progressive	16	0.7479	0.8648	+0.1169	+15.63%	6.04%	**practsig**
0.90	A	shed early	11	0.6555	0.7208	+0.0653	+9.96%	8.51%	**practsig**
0.90	A	progressive	8	0.6758	0.7596	+0.0839	+12.41%	9.64%	**practsig**
0.90	B	shed early	11	0.6555	0.7889	+0.1335	+20.37%	10.37%	**practsig**
0.90	B	progressive	7	0.6788	0.8644	+0.1856	+27.35%	10.66%	**practsig**

cell the original claim was about, no loss was found once the comparison was properly powered.

7.3 Stage 3: Bounded-Gain Configuration Selection

Exhaustive search over every LO-criticality subset, $n = 8$ tasks, 14 task sets, 6 seeds, 150,000 ticks – tractable only at this scale, which is the point: it bounds what *any* search could win over the default (execution-time-ordered shed-early) configuration.

Table 6: Exhaustive optimum against the default configuration.

Point	U	Default shed	Best shed	Gain	Verdict
A	0.70	2.8 tasks	2.6 tasks	+0.01%	not resolved
A	0.85	3.3 tasks	3.3 tasks	+0.00%	not resolved
B	0.70	2.6 tasks	2.4 tasks	+0.02%	resolved (negligible)
B	0.85	2.6 tasks	2.1 tasks	+0.79%	resolved (negligible)

The largest gain any exhaustive search found over the default, anywhere tested, is 0.79% – an order of magnitude below the 5% threshold, and this is an *upper bound*: nothing outperforms exhaustive enumeration by construction. This closes the question Phase B was built to answer: no optimisation method, however good, has more than 0.79 percentage points of service ratio available to find in this configuration space. The default configuration is adopted, and the matter is closed.

7.4 Stage 4: Budget Calibration

The regime-map effect needs no calibration – it is resolved at 30 task sets in 26 of 32 cells already. What needs sizing is the knob on the edge of resolution: shedding order. Two independent pilots of the identical comparison (by_execution_time against by_utilisation, $U = 0.7$, shed-early) illustrate why a single pilot cannot be trusted for this:

Table 7: The same comparison, two independent 24-task-set pilots.

Point	Pilot 1 effect	Pilot 2 effect	Task sets needed (Pilot 2's estimate)
A	+1.54%	+1.33%	11
B	+0.96%	+0.11%	1

Point A's two estimates agree reasonably (1.54% against 1.33%). Point B's do not: 0.96% against 0.11%, an order of magnitude apart, from two population draws of the same size and the same generation parameters. Both are below `EFFECT_FLOOR`, so the disagreement changes no conclusion – but it is exactly the instability a single small pilot cannot be trusted to report honestly, and it is the same failure mode behind the $U = 0.85$ correction in Section 7.2. `required_pairs()` applied to Pilot 2 asks for as few as 1–11 task sets to resolve *its own* observed effect at the 5% floor – which is not reassuring, since Pilot 2's effect is itself the unstable number in question. The honest conclusion is not "run a bigger ordering study"; it is the one Section 7.3 already reached by a different route: the ordering effect is too small to matter, and further calibration would only be calibrating noise.

7.5 Stage 5: Evidence-Cleared Exit

Stages 0–4 fixed *entry*: which severities to use, how many levels, which tasks to shed, in what order – and closed every one of those questions, because every knob moved the objective by under 1.6% (Section 8). None of them touched *exit*. The shipped scheme leaves a degraded level only on an idle instant, generalising AMC-RA; task_model.tex's "Mode Transition Protocol" states the exit rule as "direct or cascade, per protocol design" and leaves it open. This section is a fifth stage, added after the four above concluded, asking the question they did not: does exit timing matter as much as entry configuration turned out not to?

A confound, found before trusting the headline number. The first cut of a diagnostic measuring time spent degraded beyond what active evidence justifies returned 62–91% across the grid – implausibly large next to every other effect in this study. Tracing a real run explained why: the HI-criticality job that triggers entry typically completes quickly, but the run-queue backlog it leaves behind takes much longer to drain, and the system correctly stays degraded until that backlog clears. A busy queue draining after its trigger has gone is not a cost. The metric used below instead – LO-criticality jobs actually abandoned on release *during* that unjustified tail – is not subject to this confound: a job dropped during the tail is a countable instance of exactly the service loss a smarter exit rule could plausibly have avoided.

The reframe that made this tractable. `shed_early` (the adopted default) uses a single drop set at every level ≥ 1 – there is no intermediate level to cascade *through*, so for this policy the only lever an exit rule has is *when* the system returns to L_0, not *which* drop set applies along the way. That is exactly the question AMC-RH already answers, with a rule that already exists, is already tested, and already has a published safety argument: exit to L_0 the instant no active, eligible job's own threshold remains met, rather than waiting for the queue to idle. `simulation.multilevel` now implements this as `exit_policy="amc_rh"`, verified bit-for-bit against `engine.py`'s `AMC_RH` at $k = 2$ (`test_k2_full_drop_amc_rh_exit_reproduces_amc_rh_exactly`), and proven safe – scoped explicitly to full exit only, not partial demotion – as Corollary 2 of the safety proof.

REVIEW – Corollary 2 (`safety_proof.md`) rests on natural(t)=0 being the *exact* safe-to-exit boundary, not an approximation of it – the argument is that $R_i(\chi)$ is a worst-case bound under full interference, so a task that hasn't crossed it is already safe under full interference regardless of what happens next. Please check that reasoning independently: it is new this pass, produced by the same process that also produced (and had to retract) the whole-run TiD overclaim flagged further down. If it holds, everything downstream in this section inherits from it.

The benefit is real, resolved, and the largest exit-related effect this study has found. Every one of the 16 cells is a positive difference; 11 are practically significant, growing with utilisation and with tight deadlines, from +2.0% at $U = 0.6$ to +22–27% at $U = 0.8$–0.9. That is an order of magnitude above every entry-side knob this study closed (Section 8: $\leq 1.6\%$) and above Stage 3's exhaustive bound on drop-strategy search ($\leq 0.79\%$).

The oscillation cost is equally real, and it is not a rounding artefact. Every one of the 16 cells shows a practically significant *increase* in level transitions, 20–87%, on baselines already in the hundreds per

Table 8: Service ratio, exit_policy="amc_rh" (candidate) paired against "idle" (baseline, today's shipped behaviour), shed_early only, both operating points. $n = 24$ task sets ($n = 8$ at $U = 0.90$/tight – population scarcity, Section 7.1), 5 seeds, 200,000 ticks, $f_p = 0.2$.

U	Regime	Point	n	Idle	AMC-RH-exit	Diff	Rel.	Verdict
0.60	implicit	A	24	0.9747	0.9942	+0.0195	+2.00%	sig.
0.60	implicit	B	24	0.9837	0.9957	+0.0120	+1.22%	sig.
0.60	tight	A	24	0.9259	0.9802	+0.0543	+5.86%	**practsig**
0.60	tight	B	24	0.9373	0.9825	+0.0451	+4.81%	sig.
0.70	implicit	A	24	0.9420	0.9832	+0.0411	+4.37%	sig.
0.70	implicit	B	24	0.9601	0.9868	+0.0267	+2.78%	sig.
0.70	tight	A	24	0.8566	0.9654	+0.1088	+12.70%	**practsig**
0.70	tight	B	24	0.8711	0.9688	+0.0976	+11.21%	**practsig**
0.80	implicit	A	24	0.9057	0.9779	+0.0723	+7.98%	**practsig**
0.80	implicit	B	24	0.9289	0.9813	+0.0524	+5.65%	**practsig**
0.80	tight	A	24	0.7772	0.9502	+0.1731	+22.27%	**practsig**
0.80	tight	B	24	0.8070	0.9543	+0.1473	+18.25%	**practsig**
0.90	implicit	A	24	0.8118	0.9610	+0.1492	+18.37%	**practsig**
0.90	implicit	B	24	0.8471	0.9648	+0.1177	+13.89%	**practsig**
0.90	tight	A	8	0.7355	0.9332	+0.1977	+26.89%	**practsig**
0.90	tight	B	8	0.7948	0.9418	+0.1469	+18.49%	**practsig**

run (125–411), so the increase is substantial in absolute count too (+33 to +275 transitions), not merely a large percentage of a small number. It grows with utilisation in lockstep with the service-ratio benefit – exactly the same cells that gain the most also oscillate the most. This is the direct, measured answer to whether evidence-cleared exit trades service for churn: it does, and the trade is not free in either direction. Whether that trade is worth taking is an editorial judgement this document does not make unilaterally; δ, the fixed weight metrics_objective.md assigns level transitions in Φ, is deliberately small relative to $\alpha(U)$ and $\beta(U)$, which is this project's own prior that churn matters less than service loss and time degraded – but a 20–87% relative increase is large enough that this prior should be checked against the combined objective explicitly before the result is presented as an unqualified win, not assumed from the weight alone.

DECISION – No implementation of Φ exists in the codebase to check this against (Stages 1–4 reported service ratio directly, never a combined score). Two ways forward: (a) implement Φ with metrics_objective.md's adaptive weights and re-score every exit-policy result already measured, or (b) skip Φ and make an explicit engineering call in prose ("we accept up to X% more transitions for Y% service gain"). (a) is more defensible but is new implementation work with its own validation burden; (b) is faster but is a judgement call this document has so far declined to make unilaterally. Which do you want?

7.5.1 Tempering the churn: a hold-off, and where its sweet spot moves

The oscillation cost above motivates a tempered exit: leave a degraded level once evidence has been continuously clear for `hold_off` ticks, rather than immediately. Unlike evidence-cleared exit, a hold-off's deadline is a *timed* condition, not an event-triggered one, so realising it needed new machinery – an extension of the existing next-event lookahead (`_next_evidence_reappearance`) so the simulation loop cannot skip past evidence reappearing between otherwise-scheduled events. Two limits make `exit_policy="hysteresis"` an exact interpolation between the two policies already measured, not merely a variant of them: `hold_off=0` reproduces `exit_policy="amc_rh"` bit-for-bit, and a `hold_off` longer than any realistic gap reproduces `exit_policy="idle"` bit-for-bit – both verified in `tests/simulation/test_multilevel.py`, not argued.

Table 9: Level transitions per run – the oscillation cost, tracked deliberately rather than assumed away. Same cells, configuration and pairing as Table 8.

U	Regime	Point	Idle	AMC-RH-exit	Diff	Rel.	Verdict
0.60	implicit	A	129.1	161.9	+32.8	+25.4%	**practsig**
0.60	implicit	B	125.6	161.9	+36.3	+28.9%	**practsig**
0.60	tight	A	323.8	390.1	+66.3	+20.5%	**practsig**
0.60	tight	B	322.6	390.0	+67.5	+20.9%	**practsig**
0.70	implicit	A	166.3	222.3	+55.9	+33.6%	**practsig**
0.70	implicit	B	156.5	222.0	+65.5	+41.9%	**practsig**
0.70	tight	A	402.2	522.4	+120.2	+29.9%	**practsig**
0.70	tight	B	398.4	522.6	+124.2	+31.2%	**practsig**
0.80	implicit	A	147.8	209.6	+61.8	+41.8%	**practsig**
0.80	implicit	B	140.8	209.7	+68.8	+48.9%	**practsig**
0.80	tight	A	396.7	568.3	+171.6	+43.3%	**practsig**
0.80	tight	B	380.8	568.1	+187.3	+49.2%	**practsig**
0.90	implicit	A	185.6	300.4	+114.7	+61.8%	**practsig**
0.90	implicit	B	160.7	300.4	+139.7	+86.9%	**practsig**
0.90	tight	A	411.4	631.6	+220.2	+53.5%	**practsig**
0.90	tight	B	356.4	631.6	+275.2	+77.2%	**practsig**

Safety does not depend on `hold_off`, or on the lookahead's precision. $R_i(\chi)$ is a worst-case bound computed under full interference, so a task that has not yet crossed it is guaranteed safe under full interference from that instant on, regardless of what happens next – natural$(t) = 0$ is not a reasonable proxy for the safe-to-exit boundary, it *is* that boundary, exactly. Every exit under `"hysteresis"` is gated on a *fresh* check of this condition at the instant it fires, never on the hold-off bookkeeping alone, so an imprecise or delayed wake-up can only delay an individually-safe exit, never bring one forward into unsafe territory. This is Corollary 3 of the safety proof (`safety_proof.md`); it needs nothing beyond what Corollary 2 already established.

REVIEW – Corollary 3 leans entirely on the "fresh check gates every exit" property of `exit_if_idle` in `multilevel.py` – worth reading that function directly (not just this prose summary) to confirm the hold-off bookkeeping (`evidence_clear_since`) genuinely never substitutes for the fresh `_natural_level` check, including on the escalation-reset path. Also worth checking: does the scope restriction to full-exit-only (inherited from Corollary 2) actually still hold here, or does chaining a delay onto the exit introduce anything Corollary 2 didn't have to consider?

A real, paired sweep – `exit_opportunity.hysteresis_sweep`, not a retrospective estimate – measured `hold_off` $\in \{0, 25, 50, 100, 200, 400, 800, 1600, 3200\}$ ticks against `"idle"`, shed-early, point B, across U and both deadline regimes ($n = 16$ task sets, $n = 7$ at $U = 0.9$/tight, 5 seeds). Both curves – service-ratio gain and level-transition increase – fall as `hold_off` grows, but the transition increase falls faster, which is what makes an intermediate `hold_off` worth choosing rather than either extreme. Anchoring on the smallest `hold_off` that caps the level-transition increase at $\leq 10\%$:

The sweet spot moves with utilisation, and not in the forgiving direction. At $U = 0.6$–0.7 a modest hold-off (200–400) caps oscillation cheaply, keeping 28–43% of the gain. At $U = 0.9$ the same 10% cap needs a hold-off 2–8× larger and keeps only $\approx 10.5\%$ – churn is denser at high utilisation (baseline level-transition counts roughly double from $U = 0.6$ to $U = 0.9$ in the same regime), so a fixed hold-off

Table 10: Smallest hold_off capping the level-transition increase at 10%, and the fraction of the hold_off=0 service-ratio gain retained at that point.

U	Regime	hold_off for LT $\leq$ 10%	Service-ratio gain retained
0.60	implicit	400	28.1%
0.60	tight	200	39.2%
0.70	implicit	400	31.5%
0.70	tight	200	43.4%
0.80	implicit	400	35.8%
0.80	tight	400	19.6%
0.90	**implicit**	**1600**	**10.5%**
0.90	**tight**	**800**	**10.5%**

suppresses proportionally less of it. There is no single hold-off that is simultaneously cheap and effective across the whole grid tested; any deployed value is a choice about which utilisation regime to favour, not a free parameter fixed once. An interactive rendering of the full sweep (both metrics, both regimes, all four U values) is available alongside this project's supporting documentation (`docs/exit_strategy_analysis.md`, "Result 4").

DECISION – The data suggest `hold_off` shouldn't really be one fixed constant – it wants to track something like current utilisation. Worth exploring: could `hold_off` be derived at runtime from an observable proxy (recent trigger rate, occupancy) rather than fixed at deploy time? That's unimplemented and unmeasured – a genuine next phase, not a small addition. Separately, and smaller: this sweep only covers operating point B (termination); point A (conservative) hasn't been swept, so it isn't known whether the same U-dependence shows up there.

Progressive: cascade adds almost nothing beyond full exit alone. The diagnostic upper bound (before the mechanism existed) suggested progressive's opportunity might exceed shed_early's. Decomposing it by what kind of exit each unjustified drop would have needed answers whether that headroom is real: of the same JNE-during-tail measure, 92–97% is recoverable by full exit alone in every cell tested, leaving a cascade-specific remainder of 0.04–0.86 percentage points – itself below this study's 5% floor everywhere, and an order of magnitude below it in most cells. Intuitively this is because a shallow rung's evidence typically has a head start on clearing: by the time the deepest rung's evidence has gone stale, the shallow rung's usually has too, so the system needs a partial demotion (not a full exit) only rarely. **This closes the cascade question with evidence, the same way Stage 3 closed the drop-strategy search**: a general cascade-exit mechanism would need a new abstraction in `multilevel.py` (direct exit is currently the only exit rule it supports) and a new safety argument (unlike full exit, an intermediate level's admissibility was certified for a system that *escalated* to it, not one that *demoted* to it – Corollary 2's scope note), and the table above shows that investment has almost nothing left to buy. It is not recommended.

REVIEW – Lower priority than the two notes above, but worth a quick sanity check: does 0.04–0.86pp of headroom, consistently below the 0.79pp Stage 3 found for drop-strategy search, actually read as conclusive to you, or does it want a slightly larger population at the hardest cells (where progressive's own n is already thin) before calling it closed? This is the kind of small-effect claim the project has been burned by before (the U=0.85 retraction).

Safety. Corollary 2 of the safety proof establishes that evidence-cleared exit changes none of Theorem 1, the LO-deadline corollary, Lemma 1, or Corollary 1' – full exit to L_0 inherits AMC-RH's own existing safety argument rather than needing a new one, because L_0 is unrestricted normal mode under both schemes. It is explicitly *not* claimed that total time degraded can only fall over a whole run: admitting a LO job idle-exit would have dropped can, via priority interference, pull a later HI-criticality task's inherited busy-period start earlier and trigger an escalation the idle-exit run would not have had. Safety does not depend on this either way; net TiD, service ratio and oscillation are exactly the measured quantities in Tables 8 and 9, not derived from the proof.

REVIEW – This paragraph records a correction: an earlier draft of Corollary 2 claimed whole-run TiD can only decrease under evidence-cleared exit, which is not proven and is not true in general (the priority-interference mechanism above). It was caught while writing the proof, not after. Worth treating as a flag on this whole line of proof work rather than a closed matter – if one overclaim slipped through a first pass, a second pass (yours) is the right way to find out whether there is another.

7.6 Status of the original phases

Section 7.7 (landscape characterisation) is superseded by Stage 0 and Stage 1: it characterised severity, which is now known to be vacuous under the endorsed policy. Section 7.8 (method comparison) is retired for the reason above. Section 7.9 (budget calibration) survives intact as Stage 4, and its seeds-versus-task-sets finding still holds – it is a property of the ensemble's variance structure, not of what is being compared. They are retained below as the record of what was planned and why it changed.

7.7 Phase A: Landscape Characterisation

Question. How does the objective actually vary with severity, and how much of that variation is signal versus noise at a practical seed budget?

Method. Sweep a coarse severity grid against a small, diverse task set ensemble (spanning the utilisation range this work studies), holding the seed block fixed across every grid point (common random numbers). This does not yet require the k-level engine: the severity-indexed trigger $R_i(\chi)$ (severity-indexed response time, task_model.tex) can be dropped into the existing two-level engine as a single threshold, in place of R_i^{lo}, isolating one dimension of the eventual landscape – trigger *frequency* as a function of severity – without yet modelling a graded operating budget or a nested drop set. That is a real limitation, not a simplification of convenience: it characterises when a level would fire, not what Φ would be once it does, so it is necessary but not sufficient evidence for Section 4.

Pilot (executed). 16 task sets (4 at each of $U \in \{0.6, 0.7, 0.8, 0.9\}$, $n = 20$ tasks each, R1-schedulable), $\chi \in \{0, 0.05, 0.1, 0.2, 0.3, 0.5, 0.7, 1.0\}$, 24 seeds per (task set, χ) pair, 10^6-tick horizon scaled to $4{\times}10^5$ ticks, $f_p = 5{\times}10^{-3}$: 3,072 simulation runs in 84 s (≈ 27 ms/run).

Table 11: Trigger frequency (NiD) and abandoned jobs (JNE) vs. severity, mean over the ensemble ± standard error

χ	mean NiD	mean JNE	mean TiD
0.00	7.33 ± 1.55	3.87 ± 0.71	0.00100 ± 0.00014
0.05	6.91 ± 1.49	3.31 ± 0.64	0.00085 ± 0.00013
0.10	6.60 ± 1.47	2.85 ± 0.59	0.00074 ± 0.00012
0.20	5.30 ± 1.14	1.90 ± 0.45	0.00046 ± 0.00009
0.30	4.60 ± 1.05	1.21 ± 0.33	0.00031 ± 0.00007
0.50	3.54 ± 0.93	0.41 ± 0.15	0.00011 ± 0.00003
0.70	1.39 ± 0.45	0.08 ± 0.04	0.00002 ± 0.00001
1.00	0.00 ± 0.00	0.00 ± 0.00	0.00000 ± 0.00000

Findings. (i) Mean NiD is non-increasing across all seven consecutive pairs of grid points – zero violations – which is an observable, testable consequence of property (B) (severity-indexed response time, task_model.tex) rather than an independent discovery, and it is reassuring that the data does not contradict it. (ii) At $\chi = 1$, NiD is exactly zero for every task set in this ensemble at this seed budget: charging every task uniformly at C_i^{hi} is conservative enough that, for an R1-schedulable population not further filtered for being AMC-rtb non-trivial, the deepest severity is rarely or never reached inside a 4×10^5-tick horizon. (iii) The between-task-set coefficient of variation at $\chi = 0.1$ is **0.89** – larger than the per-seed noise reported in Section 2 – confirming that task-set-to-task-set variation, not Monte Carlo noise within a task set, is the dominant source of spread in this comparison, which is what Section 7.9 exploits directly.

Exit criterion. A characterisation this coarse cannot certify smoothness, only fail to contradict it; the finding that matters for Section 7.8 is that the signal (a several-fold change in NiD across the grid) is

large relative to the per-point standard error, so a method comparison built on this ensemble is comparing something real, not noise.

7.8 Phase B: Method Comparison

Question. Under a matched evaluation budget, does any candidate method from Section 3 find a materially better configuration than exhaustive enumeration, or find a comparable one for less?

Method. Fix a total evaluation budget equal to what exhaustive enumeration spends at a chosen k (lattice size $\times$ seed budget from Section 7.9). Give a genetic algorithm and a random-search baseline the identical budget, each repeated independently several times with different internal random seeds – the metaheuristics carry their own randomness on top of the simulator's, and a single run of either cannot distinguish a good method from a lucky one. Report, per method: regret against the enumeration's known optimum (enumeration is exhaustive, so it is the ground truth this comparison needs, not an estimate); the variance of the reported optimum across the method's own repeated runs; and wall-clock cost.

Status. Not yet executed. Section 7.7's pilot uses the existing two-level engine as a stand-in for a single trigger; a real comparison needs Φ from an actual k-level run (graded operating budget, nested drop set), which needs the k-level engine this project has not yet built. This phase is specified precisely so it can run as soon as that engine exists, rather than being designed after the fact once it is built.

7.9 Phase C: Budget Calibration

Question. How many task sets and how many seeds per task set does the eventual comparison actually need – enough for the study's stated 5% practical-significance threshold to be resolvable, not more than that?

Method. Run a small pilot, aggregate to one value per task set (`aggregate_by_taskset`), and feed the pilot into `required_pairs()` (`amc_tasksim.experiments.contract`) to estimate the ensemble size the target effect needs. Then confirm the prediction empirically – rerun at the predicted scale and check the standard error actually narrowed as predicted – *before* committing the full study's compute budget to it.

Pilot (executed). Comparing two adjacent, hard-to-separate severities from Section 7.7 ($\chi = 0$ vs. $\chi = 0.05$) on JNE, with the same 16-task-set ensemble and 10 seeds per task set:

```
n=16  diff=-0.469 (-12.8%)  95% CI [-0.652, -0.286]  resolvable=5.0%  practically
significant
```

`required_pairs()` against a 5% target effect returned **54** task sets, against the pilot's 16.

The confirmation step did not go as expected, and that is the finding. The first attempt scaled the wrong axis – seeds per task set, 10 $\rightarrow$ 40 – expecting the standard error to roughly halve ($1/\sqrt{4}$). It did not: the ratio measured was 0.92$\times$, not 2$\times$. Re-running instead with `required_pairs()`'s actual axis, task-set count (16 $\rightarrow$ 32 $\rightarrow$ 64, seeds held at 10), gave resolvable bounds of 5.0% $\rightarrow$ 6.4% $\rightarrow$ 4.5% – noisy, but tracking in the right direction, unlike the seed-count attempt. Section 7.7's between-task-set CV of 0.89 explains why directly: when between-task-set variance dominates Monte Carlo noise within a task set, adding seeds to each task set narrows only the smaller component, and the paired standard error barely moves. **The lever that narrows it is more task sets, not more seeds per task set** – the opposite of the first, more natural-seeming instinct, and not something the structural argument in Section 4 would have surfaced on its own.

Exit criterion. A calibrated (ensemble size, seed count) pair, confirmed by rerunning at that scale and checking the resolvable bound against the target – not merely computed once and trusted.

8 What This Protocol Does and Does Not Yet Establish

This section is written after Section 7, and supersedes the version of it written before the k-level engine existed. The original question – does a metaheuristic beat exhaustive enumeration at finding good trigger severities? – has been answered, but not by running that comparison. It has been answered by showing the comparison is not worth running: trigger spacing has no effect on the endorsed (shed-early) policy at all,

and every knob that survives Stage 0's pruning moves service ratio by under 1.6% against this study's own 5% threshold, with Stage 3's exhaustive bound placing at most 0.79% on the table for *any* method to find. Enumeration does not need to be shown to beat a genetic algorithm when the prize either method could win is an order of magnitude below what this study calls a result.

What *is* established, and at a scale the original three-phase design did not anticipate needing: the scheme beats two-level AMC-RA in all 32 configurations of the regime map, by a widening margin as utilisation rises and deadlines tighten (+2.4% to +27.3%), with 22 of those 32 results practically significant by this study's own threshold. That is the actual positive result this line of work set out to find, and it did not require optimising trigger spacing to get it – it came from the choice of drop policy (shed-early over progressive-by-default; progressive where feasible) and operating point (conservative versus termination), which Stage 1 and Stage 2 characterise directly.

Stage 5 adds a second positive result Stages 0–4 were not designed to look for: evidence-cleared exit beats the shipped idle-only exit on service ratio in 16 of 16 cells tested, 11 practically significant, up to +27% – the largest exit-related effect this study has measured, and provably safe for the adopted shed-early policy by inheritance from AMC-RH's own existing argument (Corollary 2), not a new one. It comes with a real, equally well-resolved cost: level transitions rise 20–87% across the same cells, growing alongside the benefit rather than independently of it. Whether the net trade is worth taking against the full objective Φ is not yet settled here (Section 7.5); what is settled is that neither number can be reported without the other. A tempered variant answers a narrower question fully: Section 7.5.1 shows a real sweet spot exists (churn falls faster than benefit as the hold-off grows), proves it safe for any hold-off by the same inheritance argument (Corollary 3), and shows that sweet spot is not one fixed value – it needs a hold-off 2–8× larger at $U = 0.9$ than at $U = 0.6$–0.7 to hold the same oscillation cap, retaining a correspondingly smaller share of the gain. Progressive's cascade-exit question, by contrast, *is* settled: decomposing the same measurement shows a real cascade mechanism could add at most 0.86 percentage points beyond full exit alone, in every cell tested – below threshold everywhere, closing that question the same way Stage 3 closed drop-strategy search, without needing to build or prove the mechanism.

Three things are not yet established and are named rather than glossed over. First, the regime map's grid is coarse ($U \in \{0.6, 0.7, 0.8, 0.9\}$, two deadline regimes, one progressive spacing) – it is not a claim that every point in the continuous $(U, D_i/T_i, CF, f_p)$ space behaves this well, only that no point tested behaves badly, which is itself a correction of an earlier, narrower claim to the contrary (Section 7.2). Second, progressive feasibility falls as low as 50% at the hardest utilisation and deadline combinations tested, and lower still where the population itself becomes scarce ($U = 0.9$, tight); the regime map's progressive rows are therefore conditioned on a shrinking, and not necessarily representative, subset of the population as the regime hardens. Third, Stage 5's net verdict on evidence-cleared exit is unresolved by design, not by omission: it reports the two components of the trade honestly rather than combining them into a single number this study has not validated a weighting for. All three are stated as scope, not as doubt about the results reported within that scope.

9 Review Notes and Open Decisions

This section exists only for this review pass. Remove it, along with the `\reviewnote`/`\decisionnote` *macro definitions and every call to them above, once the items below are resolved.*

The highlighted boxes throughout Sections 7.5 and 7.5.1 (**REVIEW** and **DECISION**) each attach to a specific location. A few items don't attach to one place, or came up in discussion around this draft rather than in the text itself; they are collected here instead of forced into a box somewhere.

9.1 Where to concentrate review time

Highest value first: the two safety corollaries (Corollary 2, Corollary 3 – `safety_proof.md`) and the mechanism code they justify (`exit_if_idle`, `_natural_level`, `_next_evidence_reappearance` in `multilevel.py`). These are the highest-stakes additions this pass produced – safety arguments about a real-time scheduling system – and this is exactly the category where the project has a track record of catching real mistakes, including one caught during this pass itself (the whole-run TiD overclaim, flagged inline above). The *mechanism* carries strong automated assurance already: `hold_off=0` and `hold_off`$\to \infty$ are verified *bit-identical*

to `"amc_rh"` and `"idle"` respectively, not merely similar. The *proof reasoning* has had one careful pass and one self-correction, which is evidence of care but not a substitute for an independent read.

Lower priority: the experiment functions, the reconciliation docs, and the measured data. These are backed by 361 passing tests including the equivalence checks above, and are lower-consequence if something is off – a measurement bug shows up as an implausible number, not a safety hole.

9.2 Untested combinations (proof coverage, not measurement coverage)

Corollary 2 and Corollary 3's safety arguments are stated generally (they extend to $x_{\mathrm{LO}} > 0$ via Corollary 1'(a)), but no empirical trial has actually run `"amc_rh"` or `"hysteresis"` with $x_{\mathrm{LO}} > 0$ – every measured result in Section 7.5 uses $x_{\mathrm{LO}} = 0$. If LO-criticality triggering is a direction this project wants to keep alive, that combination is unmeasured, not just unreported.

9.3 Decisions collected from elsewhere in this draft

- **The Φ net-verdict** (flagged in Section 7.5): implement Φ and re-score, or make an explicit engineering call in prose. Blocks a single recommended configuration for evidence-cleared exit.
- **Adaptive `hold_off`** (flagged in Section 7.5.1): the sweet spot's dependence on U suggests `hold_off` might want to be runtime-derived rather than fixed at deploy time. Unscoped, unimplemented – a candidate for a genuine next phase of work, not an afternoon's addition.
- **Operating point A for the hysteresis sweep**: only point B (termination) was swept; point A (conservative) is untested.

9.4 Explicitly not recommended, but worth confirming you agree

Extending `"amc_rh"`/`"hysteresis"` to the `progressive` policy. The cascade-headroom decomposition (Section 7.5) already suggests very little is left to gain there, on top of the open carry-in question that would make it unsafe to deploy without new proof work regardless. If you disagree with deprioritising this, say so here rather than downstream.

9.5 Housekeeping, not physics

This draft's new content (all of Section 7.5 and this section) was written in one extended session and compiles cleanly, but has not had the iterate-and-tighten prose pass the rest of this paper has had over time. Notation, redundancy between paragraphs, and whether Section 7.5.1 is better as its own `\subsection` than a `\subsubsection` of Stage 5 are all fair game.

References

[1] Michael C. Fu. Feature article: Optimization for simulation: Theory vs. practice. *INFORMS Journal on Computing*, 14(3):192–215, 2002.

[2] S. Gordon and D. Whitley. Serial and parallel genetic algorithms as function optimizers. In *Fifth International Conference on Genetic Algorithms*, pages 177–183, 1993.

[3] Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998.

[4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[5] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.

[6] J. Tate, B. Woolford-Lim, I. Bate, and X. Yao. Comparing design of experiments and evolutionary approaches to multi-objective optimisation of sensornet protocols. In *10th IEEE Congress on Evolutionary Computation*, pages 1137–1144, 2009.

[7] K. Whitely. A genetic algorithm tutorial. Technical Report CS-93-103, 1993.